Machine Learning Foundations

Chapter 0: Complete Study Notes

What ML is · Classical algorithms · Data augmentation · Splits · Loss & learning · Evaluation · Perceptron & MLP

AA-5750 Contemporary Issues in Analytics · Chandra Prakash Bathula

Contents

1 What is Machine Learning? 3

1.1 Programming the rules vs. learning the rules . . . 3
1.2 Machine learning in one line: find f so that $f(\mathbf{x}) \approx y$ . . . 3
1.3 What machine learning does: the three paradigms . . . 4
1.4 How machine learning does it: the learning loop . . . 4
1.5 Why do we need machine learning? . . . 5
1.6 Detailed use cases . . . 5
1.7 The machine learning workflow . . . 6
1.8 The real goal is generalisation, not memorisation . . . 7

2 Classical Machine Learning Algorithms 7

2.1 Linear Regression . . . 8
2.2 Logistic Regression . . . 10
2.3 k-Nearest Neighbours (k-NN) . . . 11
2.4 Naive Bayes . . . 12
2.5 Support Vector Machines (SVM) . . . 14
2.6 Decision Trees . . . 15
2.7 Random Forest . . . 16
2.8 Gradient Boosting and XGBoost . . . 16
2.9 K-Means Clustering . . . 18
2.10 DBSCAN . . . 19
2.11 Hierarchical (Agglomerative) Clustering . . . 20
2.12 Head-to-head: performance analysis . . . 21
2.13 Choosing an algorithm . . . 22

3 Data Augmentation 22

3.1 Why we need it: labelled data is the bottleneck . . . 22
3.2 Image augmentation . . . 23
3.3 Text augmentation . . . 23
3.4 The golden rule: would a human give it the same label? . . . 24
3.5 Where augmentation lives in the pipeline . . . 24
3.6 Does it work? A measured experiment . . . 24

4 Train, Validation and Test Splits **26**
4.1 Why three sets? . . . 26
4.2 Cross-validation . . . 26
4.3 Doing it right . . . 27

5 Loss Functions, Learning and Optimisation **27**
5.1 Loss: a single number for "how wrong" . . . 27
5.2 Learning: gradient descent . . . 28
5.3 Reading learning curves . . . 29
5.4 Regularisation . . . 29

6 Evaluating Classifiers: Confusion Matrices and Metrics **29**
6.1 The confusion matrix . . . 29
6.2 The metrics, from the matrix . . . 30
6.3 Thresholds: precision vs. recall . . . 30
6.4 The accuracy paradox . . . 31
6.5 Which metric when? . . . 31

7 Pivoting to Deep Learning: the Perceptron and the MLP **31**
7.1 Why classical ML is not enough . . . 31
7.2 The perceptron (Rosenblatt, 1958) . . . 32
7.3 The XOR problem . . . 32
7.4 From perceptron to multi-layer perceptron . . . 33
7.5 Two ingredients that made it trainable . . . 33
7.6 The MLP you will train: $784 \to 128 \to 64 \to 10$ . . . 34

8 Summary **35**
8.1 Practice questions . . . 35

1 What is Machine Learning?

1.1 Programming the rules vs. learning the rules

For most of computing history, a programmer wrote the rules. To flag a suspicious credit card transaction you would write something like *"if the amount is above $5,000 and the country is different from the card holder's, block it"*. That works until fraudsters learn the rule, or until the pattern is too subtle to put into words: nobody can write down the rules that tell a cat photo from a dog photo.

Machine learning flips the arrangement. We give the computer **examples with answers** and it **works out the rules itself**.

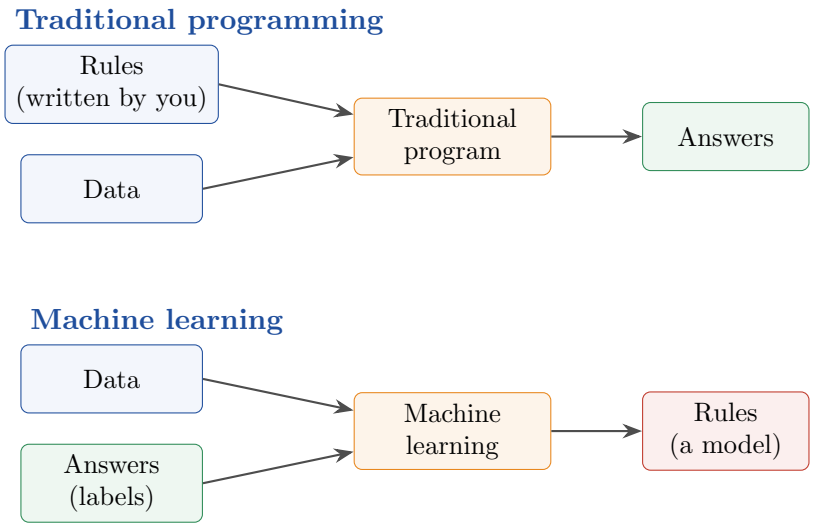


Figure 1: In ML the rules are the *output*. The learned rules (the model) are then used like any other program on new data.

Definition: Machine Learning

Machine learning is the study of algorithms that **improve at a task by learning from data** instead of following hand-written rules. Tom Mitchell's (1997) classic phrasing: a program learns from experience E with respect to a task T and performance measure P if its performance on T, as measured by P, improves with E.

Take spam filtering with Mitchell's three letters: T = label each email spam or not spam, E = a million emails that users have already marked, P = the percentage of new emails labelled correctly.

1.2 Machine learning in one line: find f so that $f(\mathbf{x}) \approx y$

Almost every supervised ML problem has the same shape:

$$\underbrace{\mathbf{x}}_{\text{input}} \xrightarrow{f_{\boldsymbol{\theta}}} \underbrace{\hat{y}}_{\text{prediction}} \qquad \text{and we want} \qquad \hat{y} \approx y \text{ (the true answer).}$$

- $\mathbf{x}$ is the **input** or **feature vector**: the pixels of an image, the words of an email, a patient's age, BMI and blood pressure.

- y is the **target** or **label**: a digit, "spam", a house price.
- $f_{\boldsymbol{\theta}}$ is the **model**, a function with adjustable knobs $\boldsymbol{\theta}$ called **parameters** (weights).

Every learning algorithm in these notes is a choice of three ingredients:

Ingredient	Question it answers	Examples
Model family	What shapes of f are allowed?	lines, trees, sets of neighbours, neural networks
Loss function	How wrong is a prediction?	squared error, cross-entropy, hinge
Optimiser	How do we turn the knobs to reduce the loss?	closed-form solution, gradient descent, greedy splitting

1.3 What machine learning does: the three paradigms

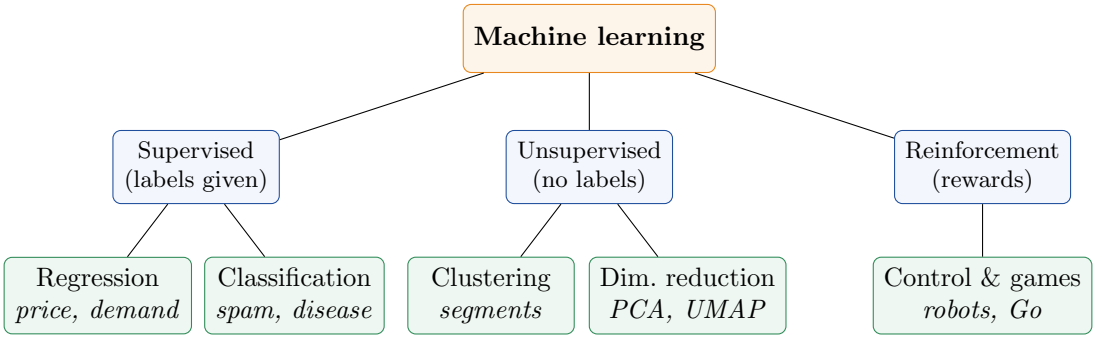


Figure 2: The three paradigms and the tasks under them.

Supervised learning. Every training example comes with the right answer. If the answer is a number (price, temperature, days until a machine fails) it is **regression**. If it is a category (spam / not spam, digit 0–9, benign / malignant) it is **classification**.

Unsupervised learning. No answers, only inputs. The algorithm finds structure on its own: groups of similar customers (**clustering**), 1,000 features squeezed into a handful that keep most of the information (**dimensionality reduction**), or points that do not fit anywhere (**anomaly detection**).

Reinforcement learning. An agent acts in an environment and receives rewards. There is no "correct label" for each step, only feedback on how well things went over time. Game-playing agents and robot control are the classic cases.

A fourth label you will hear is **self-supervised learning**: the data supplies its own labels, for example by hiding a word in a sentence and asking the model to predict it. This approach is used to pre-train language models, and we return to it later in the course.

1.4 How machine learning does it: the learning loop

Regardless of the algorithm, training follows the same loop.

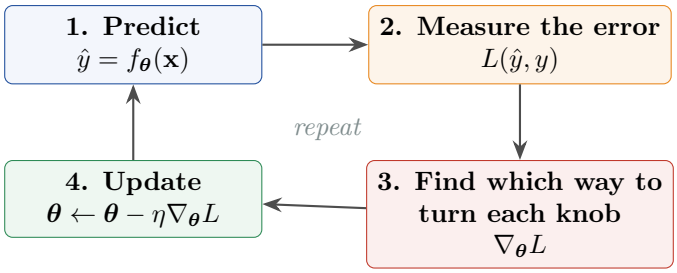


Figure 3: The universal training loop. Linear regression, logistic regression and neural networks all follow it. Trees and k-NN learn differently, but they still minimise an error.

1.5 Why do we need machine learning?

1. **The rules are too complex to write.** Recognising a face, transcribing speech or reading handwriting involves patterns no human can spell out as if/else statements.
2. **The rules keep changing.** Spammers change their wording weekly. A learned filter retrains on fresh examples; a hand-written one needs an engineer every time.
3. **The scale is beyond humans.** No analyst can personally review every one of millions of daily transactions or recommend products to every customer of a large retailer.
4. **Patterns hide in many weak signals.** A readmission risk may depend a little on age, a little on lab results and a little on the number of prior visits. Models combine dozens of weak signals better than intuition does.

When *not* to use machine learning

If the rule is simple, stable and known (tax brackets, a password check, a unit conversion), write the rule. ML adds data collection, training, monitoring and a non-zero error rate. It earns its keep only when the rules are unknown, too complex, or changing.

1.6 Detailed use cases

The table maps real problems to the vocabulary above. For each one, look at the metric column: the right way to score a model depends on what a mistake costs.

Problem	Input x	Output y	Task	Metric that matters, and why
House price estimate	size, bedrooms, location, age	price ($)	regression	RMSE or MAE in dollars: buyers care about the size of the miss
Spam filter	word counts, sender, links	spam / ham	classification	precision: a real email lost in spam is worse than one spam getting through
Cancer screening	cell measurements or a scan	malignant / benign	classification	recall: a missed cancer is far worse than a false alarm
Card fraud	amount, merchant, time, location	fraud / legit	classification (imbalanced)	recall at a fixed false-alarm rate; accuracy is useless here (Section 6.4)
Customer churn	usage, tenure, support calls	leaves / stays	classification	recall of churners: you can only save customers you flag
Demand forecast	past sales, season, promotions	units next week	regression (time series)	MAE; must be split by time, never randomly
Customer segments	purchase history	group id (unknown)	clustering	business usefulness, silhouette score
Handwritten digits	28×28 pixels	digit 0–9	classification	accuracy; classes are balanced

Worked example: a spam filter, rules vs. learning

Rule-based: "Block if the subject contains FREE, WINNER or $$$." Spammers respond with "FR EE" and "W1NNER" within days, and a genuine email titled "Free parking at the conference" is blocked.

Learned: Give a model 100,000 emails users already marked. It learns that "free" is mildly suspicious, "free" with an unknown sender and a link is very suspicious, and "free" from a colleague with "meeting" in the body is normal. When spammers change tactics, users mark new examples and the model is retrained. Nobody rewrites the rules. Naive Bayes (Section 2.4) is exactly this model.

1.7 The machine learning workflow

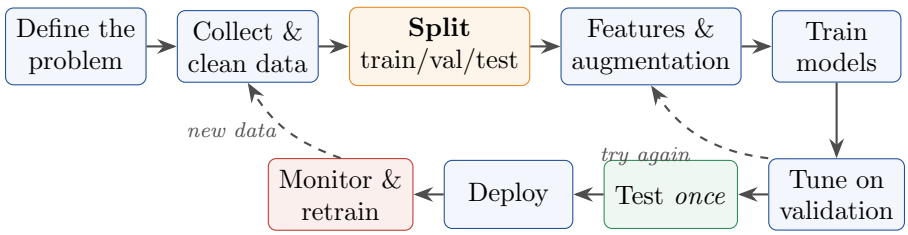


Figure 4: The end-to-end workflow. Splitting happens *before* feature engineering and augmentation so that nothing about the test set leaks into training.

1.8 The real goal is generalisation, not memorisation

A model is only useful on data it has **not seen**. A student who memorises last year's exam answers can score 100% on last year's paper and fail this year's. Models do the same.

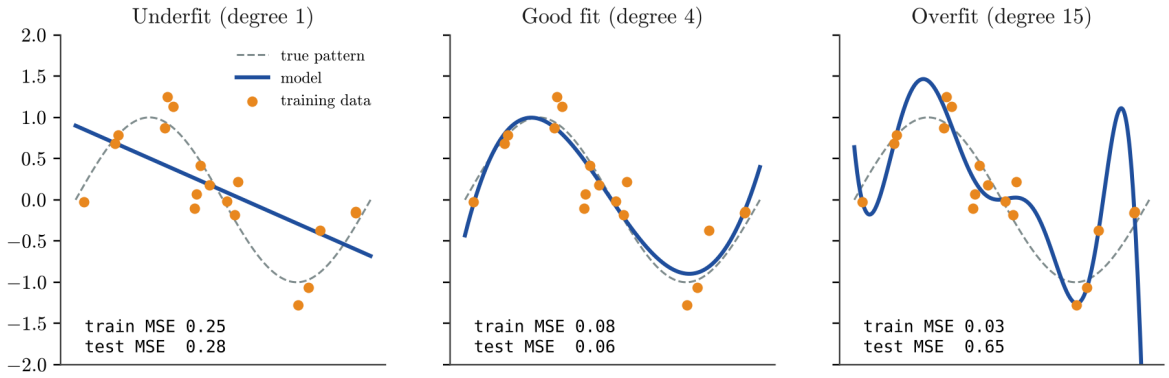


Figure 5: The same 18 noisy training points fitted with polynomials of degree 1, 4 and 15. Test MSE is measured on 200 fresh points from the same curve.

The numbers in Figure 5 tell the whole story:

Model	Train MSE	Test MSE	Diagnosis
Degree 1	0.25	0.28	**underfit**: too simple; bad on both
Degree 4	0.08	0.06	**good fit**: captures the pattern, ignores the noise
Degree 15	0.03	0.65	**overfit**: best on training, worst on new data

Definition: Bias, variance and the generalisation gap

Bias is error from a model that is too simple to represent the pattern (degree 1). **Variance** is error from a model so flexible that it chases the noise in one particular training set (degree 15); retrain it on a different sample and it looks completely different. The **generalisation gap** is test error minus training error. A large gap means overfitting.

Remember the degree-15 curve. Most of the rest of these notes is about avoiding it: validation sets, regularisation, ensembles, augmentation and early stopping are all ways of keeping variance down without pushing bias up.

2 Classical Machine Learning Algorithms

"Classical" means the algorithms that came before deep learning took over vision and language. They are still the default for **tabular data**, the rows and columns of a spreadsheet or database, which is most business data. On tables, a well-tuned gradient-boosted tree regularly beats a neural network.

Algorithm	Task	What it learns	Decision boundary	Scale features?
Linear regression	regression	one weight per feature	(a straight line / plane)	helps
Logistic regression	classification	one weight per feature	straight line	helps
k**-NN**	both	nothing; stores the data	any shape, jagged	**essential**
Naive Bayes	classification	per-class feature distributions	quadratic curves	no
SVM	classification	support vectors + weights	widest-margin line, or curved with a kernel	**essential**
Decision tree	both	if/else splits	axis-aligned boxes	no
Random forest	both	many de-correlated trees	smoothed boxes	no
XGBoost	both	trees that correct each other	boxes	no
K-Means	clustering	k centroids	round blobs	**essential**
DBSCAN	clustering	dense regions	any shape + noise	**essential**
Hierarchical	clustering	a merge tree	depends on linkage	**essential**

2.1 Linear Regression

2.1.1 Why do we need it?

Linear regression is the simplest model that learns from data, and it is still everywhere: forecasting, pricing, and above all **explaining**. Its weights say directly how much the prediction changes when a feature goes up by one unit. It is also the template for everything that follows: logistic regression is linear regression passed through a sigmoid, and a single neuron is logistic regression.

2.1.2 The model

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b$$

Each weight w_j is the change in $\hat{y}$ for a one-unit increase in x_j, holding the others fixed. The bias b is the prediction when every feature is zero.

2.1.3 How it learns: least squares

We want the line that makes the errors (residuals) $y_i - \hat{y}_i$ as small as possible overall. Squaring them makes every error positive and punishes big misses more:

$$\mathrm{MSE}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \big(y_i - (\mathbf{w}^\top \mathbf{x}_i + b)\big)^2.$$

For one feature the minimum has a closed form:

$$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad b = \bar{y} - w\bar{x}.$$

For many features (or huge datasets) we use **gradient descent**: start anywhere and repeatedly step downhill on the MSE surface. Because the MSE of a linear model is a bowl (convex), there is one minimum and gradient descent finds it.

Worked example: house prices by hand

Four houses: size x (1,000 sq ft) $= 1, 2, 3, 4$; price y (\$100k) $= 2, 3, 5, 6$.
$\bar{x} = 2.5$, $\bar{y} = 4$. $\sum(x_i - \bar{x})(y_i - \bar{y}) = (-1.5)(-2) + (-0.5)(-1) + (0.5)(1) + (1.5)(2) = 7$.
$\sum(x_i - \bar{x})^2 = 5$.
$w = 7/5 = 1.4$ and $b = 4 - 1.4 \times 2.5 = 0.5$, so $\hat{y} = 1.4x + 0.5$.
Interpretation: each extra 1,000 sq ft adds \$140k. A 2,500 sq ft house is predicted at $1.4(2.5) + 0.5 = 4.0$, i.e. \$400k.
Predictions $1.9, 3.3, 4.7, 6.1$; residuals $0.1, -0.3, 0.3, -0.1$; MSE $= (0.01 + 0.09 + 0.09 + 0.01)/4 = 0.05$.

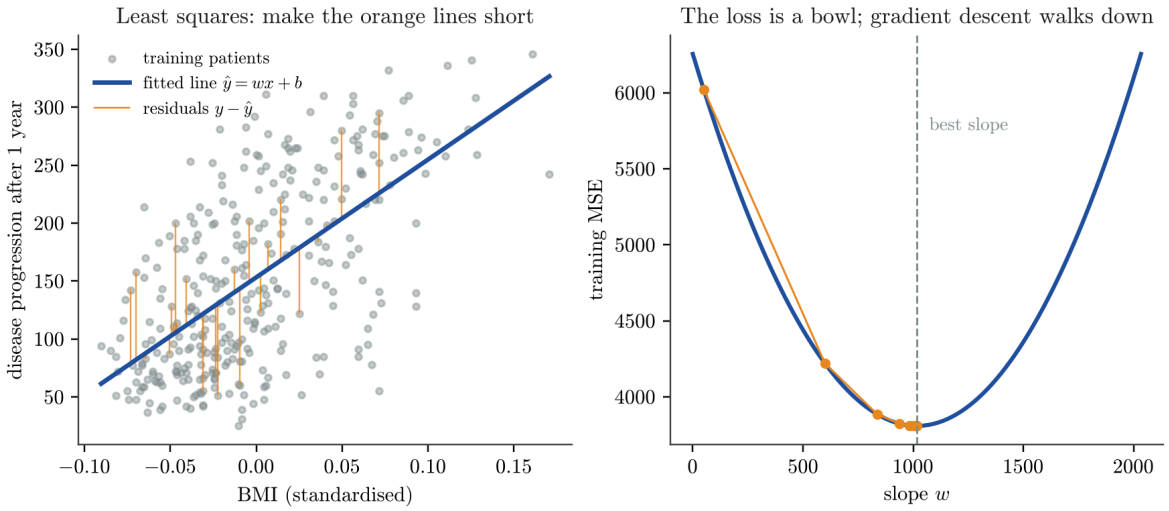


Figure 6: Left: linear regression on the diabetes dataset (442 patients), predicting disease progression from BMI. Orange lines are residuals. Right: training MSE as a function of the slope. Gradient descent (orange dots) walks down the bowl to the least-squares slope.

2.1.4 Performance on real data

On the diabetes dataset (75/25 split):

Model	Test RMSE	Test R^2
Predict the training mean for everyone (baseline)	70.5	0.00
Linear regression on BMI only	64.7	0.16
Linear regression on all 10 features	56.4	0.36

R^2 is the fraction of the target's variance the model explains: 0 is no better than predicting the mean, 1 is perfect. Going from one feature to ten more than doubles it. An R^2 of 0.36 is honest for a medical outcome: many causes of disease progression are not in the data.

Assumptions and pitfalls

- **Linearity:** if the true relationship curves, add features like x^2 or use a non-linear model.
- **Outliers:** squaring makes one wild point pull the whole line. Look at the residuals.
- **Correlated features** (multicollinearity) make individual weights unstable and hard to interpret, even when predictions are fine. Ridge regression (L2 penalty) stabilises them.
- **Correlation is not causation:** a weight says how $\hat{y}$ moves with x in this data, not what would happen if you changed x.

2.2 Logistic Regression

2.2.1 Why not just use linear regression for classification?

Code "spam" as 1 and "not spam" as 0 and fit a line, and you get predictions like -0.3 or 1.7, which are not probabilities. One extreme email also tilts the whole line. We need an output that always lies between 0 and 1.

2.2.2 What it does: a line, then a squash

Logistic regression computes the same linear score as before and passes it through the **sigmoid**:

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

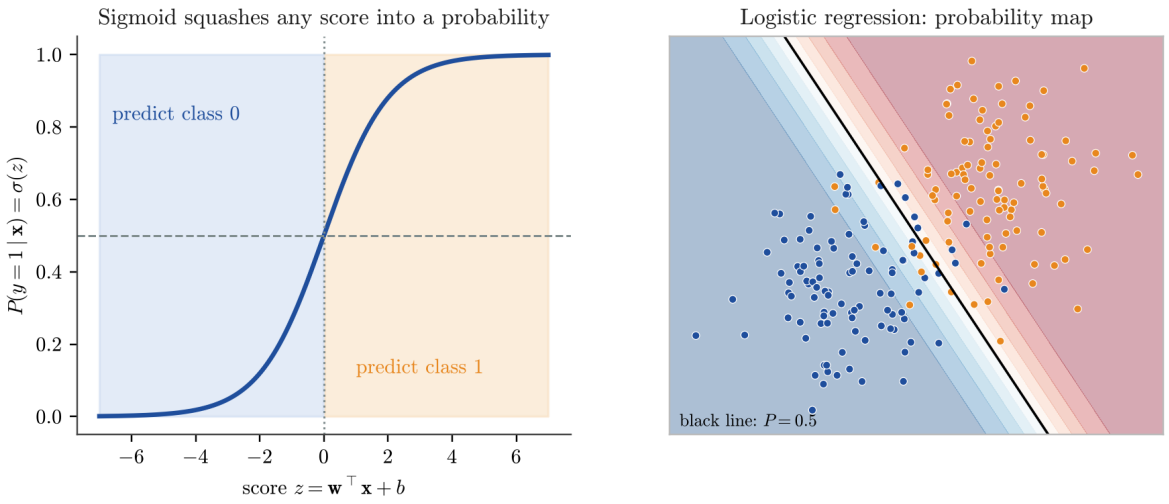


Figure 7: Left: the sigmoid turns any score into a probability; $z = 0$ maps to 0.5. Right: on two blobs, the model's probability changes smoothly across a straight decision boundary (black line, $P = 0.5$).

Intuition

Think of z as the evidence. Each feature adds or subtracts evidence according to its weight. The sigmoid turns the total into confidence: lots of positive evidence gives a probability near 1, lots of negative evidence near 0, and a balance gives 0.5. Because z is linear in $\mathbf{x}$, the set of points where $P = 0.5$ (that is, $z = 0$) is a straight line: **logistic regression is a**

linear classifier.

2.2.3 Interpreting the weights: log-odds

Rearranging the sigmoid gives

$$\log \frac{P(y=1)}{1-P(y=1)} = \mathbf{w}^\top \mathbf{x} + b.$$

The left side is the **log-odds**. So e^{w_j} is the factor by which the *odds* multiply when x_j increases by one. This is why logistic regression is the standard model in medicine and credit scoring: "each additional prior admission multiplies the odds of readmission by 1.3" is a sentence a doctor can use.

2.2.4 How it learns: binary cross-entropy

We cannot use MSE (with a sigmoid it gives a non-convex surface with flat regions). Instead we maximise the probability the model gives to the correct labels, which is the same as minimising **binary cross-entropy** (BCE):

$$L = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\Big].$$

A confident correct prediction costs almost nothing; a confident wrong one costs a lot. The gradient works out to $(\hat{p}-y)\,\mathbf{x}$, the same simple form as in linear regression. There is no closed-form solution, so it is trained with gradient descent.

Worked example: spam probability

Features: x_1 = number of times "free" appears, $x_2 = 1$ if the email contains a link. Learned weights: $w_1 = 1.2$, $w_2 = 2.0$, $b = -3$.
Email A ($x_1 = 2$, $x_2 = 1$): $z = 1.2(2) + 2.0(1) - 3 = 1.4$, so $P(\text{spam}) = \sigma(1.4) = 0.802$. Classified as spam.
Email B ($x_1 = 0$, $x_2 = 1$): $z = -1$, so $P(\text{spam}) = 0.269$. Not spam.
Odds: $e^{1.2} = 3.32$, so each extra "free" multiplies the odds of spam by about 3.3.
Loss for email A: if it really was spam, BCE $= -\ln 0.802 = 0.22$ (small). If it was a genuine email, BCE $= -\ln(1-0.802) = 1.62$ (large). The loss pushes hardest where the model is confidently wrong.

Multi-class: replace the sigmoid with a **softmax** over one score per class and BCE with categorical cross-entropy. That is exactly the output layer of the MNIST MLP in Section 7.6.

2.3 k-Nearest Neighbours (k-NN)

2.3.1 What it does

k-NN makes no model at all. To classify a new point, find the k training points closest to it and take a majority vote (for regression, average their values). "You are the average of your k closest neighbours."

2.3.2 How it does it

1. Store the whole training set. That is all the "training" there is.
2. For a query $\mathbf{x}$, compute its distance to every training point, usually Euclidean: $d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_j (x_j - x'_j)^2}$.
3. Take the k nearest and vote.

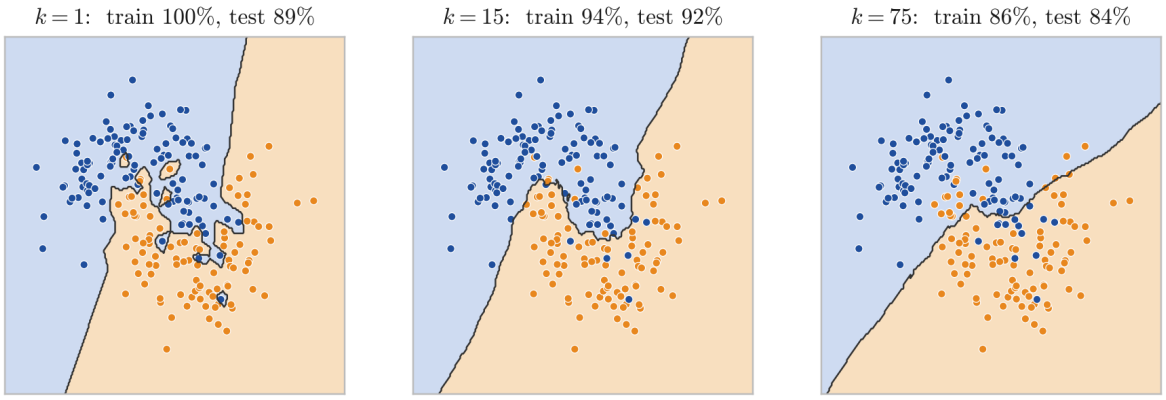


Figure 8: k controls the bias-variance trade-off. $k = 1$ memorises the training set (100% training accuracy, jagged islands, 89% test). $k = 15$ is smoother and best on test (92%). $k = 75$ is too smooth and underfits (84%).

Worked example: why scaling is essential

Fruit with features (weight in grams, sweetness on a 1–10 scale). Query: $(150, 8)$. Training: apple A $(170, 8)$, apple B $(130, 8)$, orange C $(155, 2)$.
Raw distances: A $= 20$, B $= 20$, C $= \sqrt{5^2 + 6^2} = 7.8$. 1-NN says *orange*, even though the sweetness matches the apples exactly. Grams range over tens, sweetness over single digits, so weight drowns out sweetness.
Standardised (divide weight by 20 and sweetness by 2): A $= 1.0$, B $= 1.0$, C $= 3.01$. Now both apples are closest and 3-NN votes 2–1 for *apple*.

Strengths: no training time, any boundary shape, easy to explain ("these 5 similar customers churned"). **Weaknesses:** prediction is slow (compare against every point), it needs feature scaling, and it degrades in high dimensions where all points become roughly equally far apart (the *curse of dimensionality*). **Uses:** recommendation ("customers like you"), quick baselines, imputing missing values.

2.4 Naive Bayes

2.4.1 What it does

Naive Bayes turns Bayes' rule into a classifier:

$$P(\text{class} \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid \text{class})\, P(\text{class})}{P(\mathbf{x})} \;\propto\; P(\text{class}) \prod_{j=1}^{d} P(x_j \mid \text{class}).$$

The **naive** step is the product: it assumes the features are independent of each other *within each class*. That is almost never true ("free" and "offer" tend to appear together), but the classifier

often works well anyway because it only needs to get the *ranking* of classes right, not the exact probabilities.

2.4.2 How it does it

Training is just counting. For each class, estimate the prior $P(\text{class})$ and the distribution of each feature: word frequencies for text (multinomial NB), or a mean and standard deviation per feature for continuous data (Gaussian NB). Prediction multiplies the relevant numbers and picks the largest.

Worked example: is this email spam?

From training: $P(\text{spam}) = 0.4$, $P(\text{ham}) = 0.6$. $P(\text{“free”} \mid \text{spam}) = 0.30$, $P(\text{“free”} \mid \text{ham}) = 0.02$, $P(\text{“meeting”} \mid \text{spam}) = 0.05$, $P(\text{“meeting”} \mid \text{ham}) = 0.20$.
New email contains “free” and “meeting”:

$$\text{spam score} = 0.4 \times 0.30 \times 0.05 = 0.0060$$
$$\text{ham score} = 0.6 \times 0.02 \times 0.20 = 0.0024$$
$$P(\text{spam} \mid \text{email}) = 0.0060/(0.0060 + 0.0024) = 0.714$$

“Free” is 15 times more common in spam, which outweighs “meeting” being 4 times more common in ham.
Zero problem: if “crypto” never appeared in ham during training, $P(\text{“crypto”} \mid \text{ham}) = 0$ and one word vetoes the entire email. **Laplace smoothing** adds 1 to every count, $P(w \mid c) = (\text{count}(w, c) + 1)/(N_c + |V|)$, so nothing is ever exactly zero.

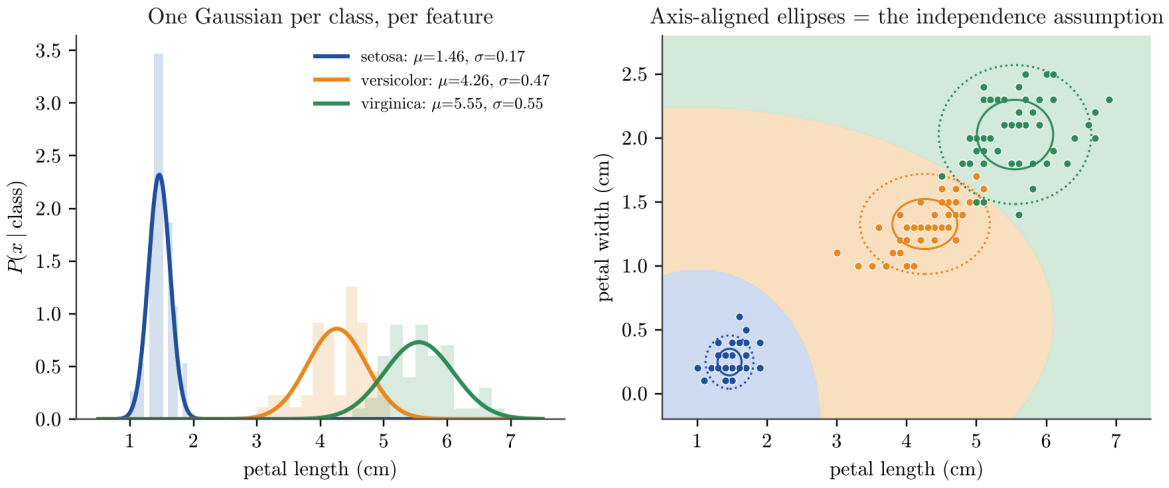


Figure 9: Gaussian Naive Bayes on iris. Left: one bell curve per class for petal length. Right: with two features, each class is an ellipse aligned with the axes (no tilt), which is what the independence assumption looks like geometrically.

Strengths: extremely fast, works with little data, handles thousands of features (text). **Weaknesses:** correlated features get double-counted. On the digits dataset neighbouring pixels are strongly correlated and Naive Bayes drops to 85% while most other models reach 97–98% (Table 1). **Uses:** spam filtering, document categorisation, sentiment baselines.

2.5 Support Vector Machines (SVM)

2.5.1 What it does

Many lines can separate two classes. The SVM picks the one with the **widest margin**: the widest "street" that has no points inside it. A wide street is less likely to be crossed by new points, so it generalises better.

2.5.2 How it does it

For labels $y_i \in \{-1, +1\}$ the SVM solves

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max\bigl(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\bigr).$$

The first term widens the margin (margin width $= 2/\|\mathbf{w}\|$). The second, the **hinge loss**, charges for points inside the street or on the wrong side. C sets the trade-off: large C = few violations allowed, narrow street, risk of overfitting; small C = wider street that tolerates some mistakes.

The points on or inside the margin are the **support vectors**. They alone define the boundary; you could delete every other training point and get the same SVM. In the left panel of Figure 10, 3 of 60 points are support vectors.

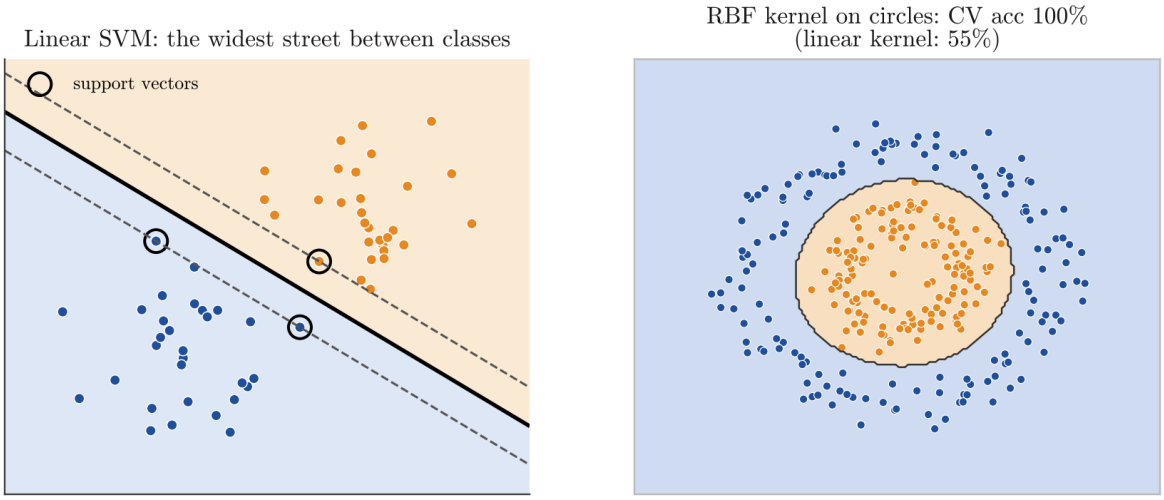


Figure 10: Left: linear SVM; the solid line is the boundary, dashed lines are the edges of the margin, circled points are support vectors. Right: concentric circles cannot be split by a line (linear SVM: 55% cross-validated accuracy) but an RBF kernel separates them perfectly (100%).

2.5.3 The kernel trick

Points on two concentric circles cannot be separated by a line in 2D. Add a third feature $x_1^2 + x_2^2$ (the squared distance from the centre) and they separate with a flat plane in 3D. The **kernel trick** lets the SVM work in such higher-dimensional spaces without ever computing the new features: it only needs similarities $K(\mathbf{x}, \mathbf{x}')$ between pairs of points. The RBF (Gaussian) kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ corresponds to an infinite-dimensional feature space.

Strengths: excellent on small-to-medium, high-dimensional data; strong theory. It had the best digits score in our benchmark (98.1%). **Weaknesses:** training scales badly beyond roughly 100k

rows, needs feature scaling and tuning of C and γ, and gives no probabilities by default. **Uses:** text classification, bioinformatics, image classification before deep learning.

2.6 Decision Trees

2.6.1 What it does

A decision tree is a flowchart of yes/no questions learned from data: "Is petal width ≤ 0.8 cm? If yes, setosa. If no, is petal width ≤ 1.75 cm?" It is the model closest to human reasoning and the building block of random forests and XGBoost.

2.6.2 How it does it: greedy splitting on impurity

At every node, try every feature and every threshold, and keep the split that makes the two children **purest**. Purity is usually measured with Gini impurity:

$$G = 1 - \sum_{c} p_c^2,$$

where p_c is the fraction of class c in the node. $G = 0$ means the node holds a single class. Repeat on each child until the nodes are pure or a stopping rule (maximum depth, minimum samples) kicks in.

Worked example: the first split on iris

Root: 50 setosa, 50 versicolor, 50 virginica, so $G = 1 - 3 \times (1/3)^2 = 0.667$.
Split "petal width ≤ 0.8": the left child is 50 setosa ($G = 0$); the right child is 0/50/50 ($G = 1 - 0.5^2 - 0.5^2 = 0.5$).
Weighted impurity after the split $= \frac{50}{150}(0) + \frac{100}{150}(0.5) = 0.333$, an impurity drop of $0.667 - 0.333 = 0.333$. No other split does better, so this one is chosen.
Next split "petal width ≤ 1.75" gives a node with 0/49/5: $G = 1-(49/54)^2-(5/54)^2 = 0.168$, nearly pure. These numbers appear in Figure 11.

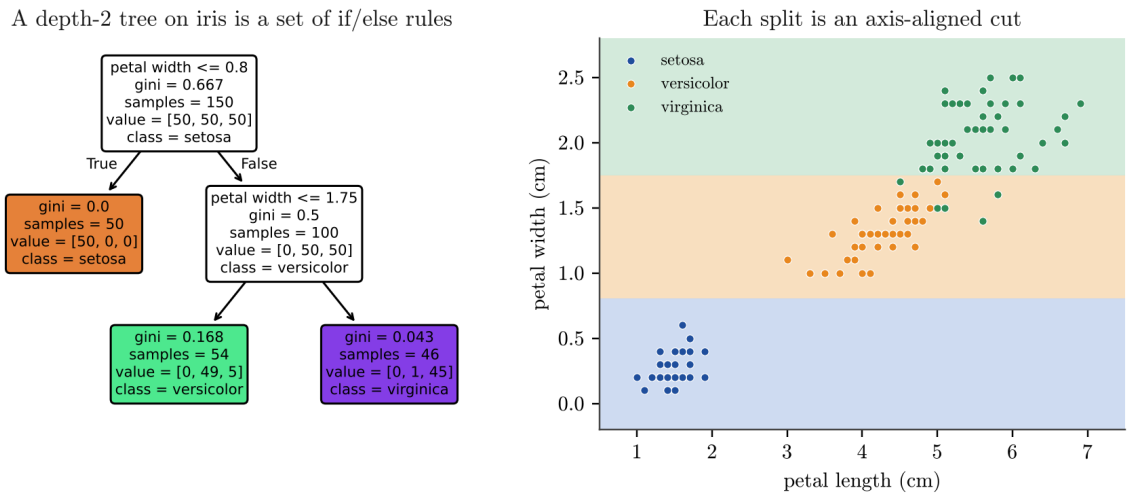


Figure 11: A depth-2 tree on iris, drawn by scikit-learn. Each split is a cut parallel to an axis, so the regions are rectangles.

Strengths: readable, no scaling needed, handles mixed numeric and categorical data, captures interactions. **Weaknesses:** a deep tree memorises (100% training accuracy on moons in Figure 12), and small changes in the data can produce a completely different tree. That instability is exactly what ensembles fix.

2.7 Random Forest

2.7.1 Why: the wisdom of crowds

One deep tree has low bias and very high variance. Averaging many trees keeps the low bias and cancels much of the variance, *provided the trees make different mistakes*. Random forests make them different in two ways.

2.7.2 How it does it

1. **Bagging (bootstrap aggregating):** each tree trains on a bootstrap sample, n rows drawn *with replacement*. Each tree sees about 63% of the distinct rows.
2. **Feature subsampling:** at each split, a tree may only consider a random subset of features (typically $\sqrt{d}$ for classification). This stops every tree from splitting on the same dominant feature.
3. **Vote:** classification takes the majority vote across trees; regression averages them.

Bonus: the ≈37% of rows each tree never saw form a free **out-of-bag** validation set, and the forest reports **feature importances** for free.

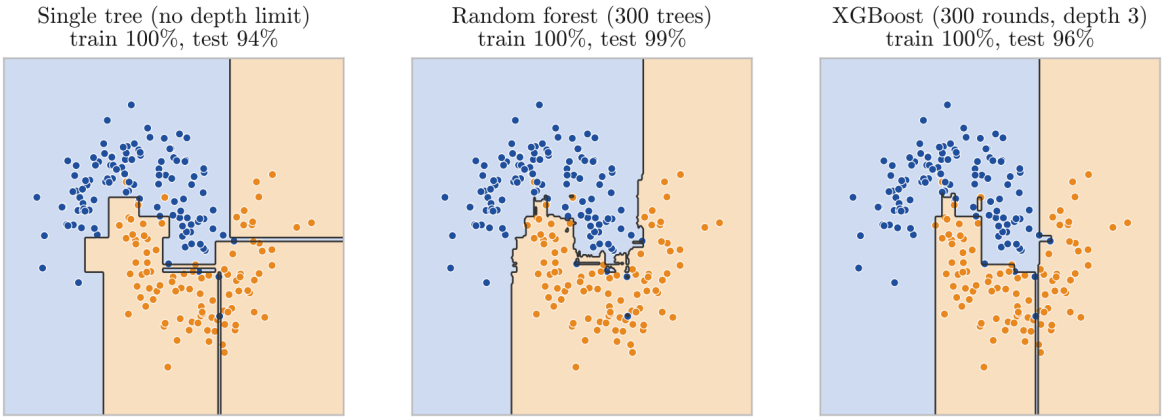


Figure 12: Three tree-based models on noisy moons (210 training points, 90 test points). The single tree carves narrow slivers around individual points. The forest's averaged boundary is smoother and scores best on test (99% vs. 94%).

On the digits benchmark (Table 1) a single tree scores 85.9%; a forest of 300 trees scores 97.7%. That 12-point jump is variance reduction, with no change to the base learner.

2.8 Gradient Boosting and XGBoost

2.8.1 Bagging vs. boosting

A random forest builds trees *independently* and averages them. **Boosting** builds them *in sequence*: each new tree is trained to fix the errors of all the trees so far. Forests reduce variance; boosting mostly reduces bias, using small, weak trees.

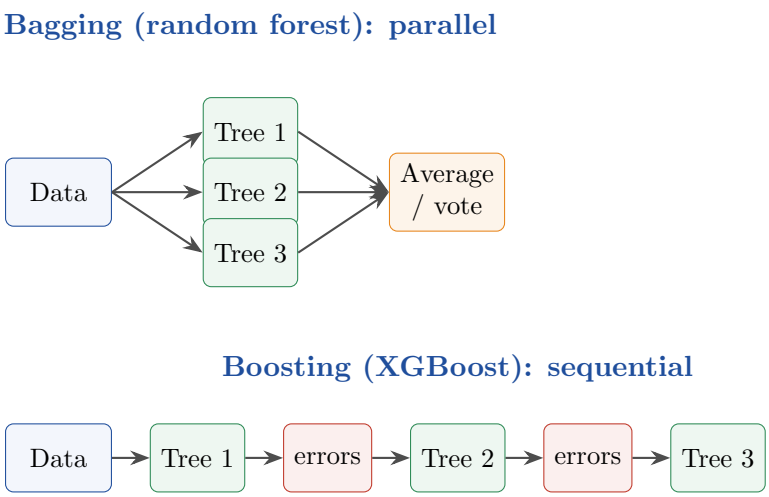


Figure 13: Bagging trains trees independently on resampled data. Boosting trains each tree on what the previous trees still get wrong.

2.8.2 How gradient boosting works

1. Start with a constant prediction F_0 (the mean, or the log-odds of the positive class).
2. Compute the **residuals** (more generally, the negative gradient of the loss) for every row.
3. Fit a small tree h_m to those residuals.
4. Update $F_m = F_{m-1} + \eta\, h_m$, with learning rate η around 0.05–0.3.
5. Repeat for M rounds.

Worked example: three houses

Prices (\$k): 200, 300, 400. Start with $F_0 = 300$ (the mean). Residuals: $-100, 0, +100$. Tree 1 learns those residuals. With $\eta = 0.1$: $F_1 = 290, 300, 310$. New residuals: $-90, 0, +90$. Each round closes 10% of the remaining gap. The small learning rate means many cautious steps, which generalises better than a few big ones.

2.8.3 What XGBoost adds

XGBoost (Chen & Guestrin, 2016) is gradient boosting engineered for speed and accuracy: it uses second-order gradient information, adds L1/L2 penalties on the leaf values (regularisation built into the objective), handles missing values natively, subsamples rows and columns, and parallelises the split search. It and its relatives (LightGBM, CatBoost) dominate competitions on tabular data.

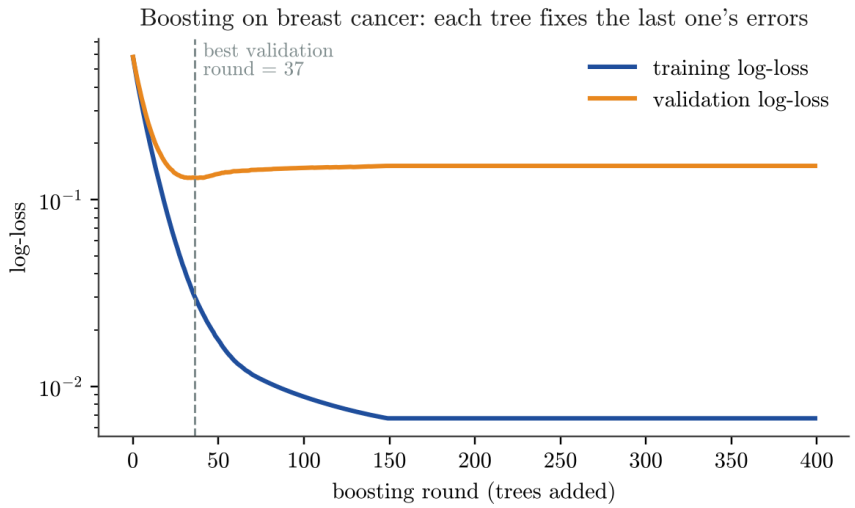


Figure 14: XGBoost on breast cancer. Training loss keeps falling with every tree, but validation loss bottoms out at round 37 (0.130) and creeps up to 0.151 by round 400. Boosting can overfit, so use early stopping on a validation set.

2.9 K-Means Clustering

2.9.1 What it does

K-Means splits unlabelled data into k groups so that each point is close to its group's centre (centroid). The business version: "split our customers into 4 segments so that customers within a segment look alike."

2.9.2 How it does it (Lloyd's algorithm)

1. Pick k initial centroids (k-means++ chooses them spread out).
2. **Assign** every point to its nearest centroid.
3. **Update** every centroid to the mean of its points.
4. Repeat 2–3 until the assignments stop changing.

It minimises the **inertia**, the within-cluster sum of squared distances $\sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|^2$. Every step lowers it, so it always converges, though possibly to a local minimum. That is why libraries run it from several starts (`n_init`).

Worked example: one-dimensional K-Means

Points: 1, 2, 3, 10, 11, 12. $k = 2$, initial centroids 0 and 7.
Assign: 1, 2, 3 are closer to 0; 10, 11, 12 are closer to 7. **Update:** centroids become $(1+2+3)/3 = 2$ and $(10+11+12)/3 = 11$. **Assign again:** nothing changes. Converged with inertia $= 1+0+1+1+0+1 = 4$.

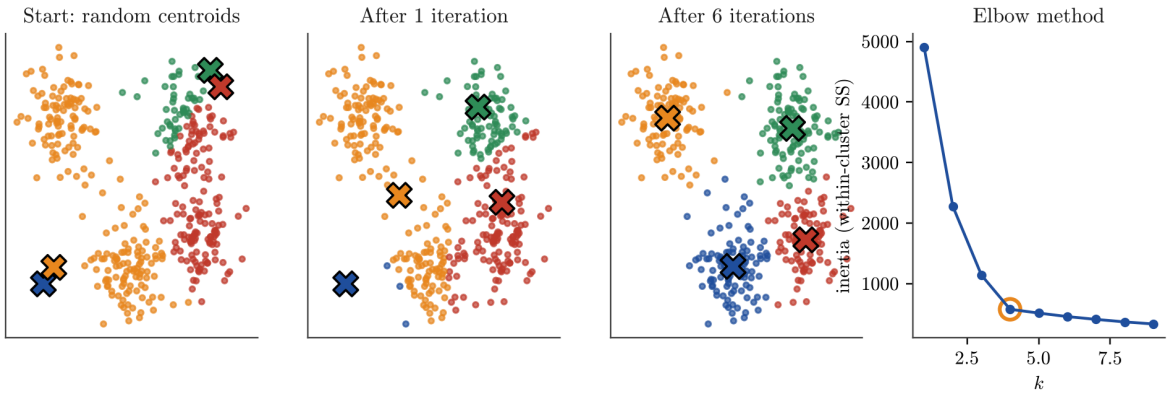


Figure 15: K-Means on four blobs from deliberately poor starting centroids. After six iterations it has found the four groups (adjusted Rand index 0.94 against the true groups). Right: the elbow method. Inertia always drops as k grows; the bend at $k = 4$ is where extra clusters stop paying off.

Weaknesses: you must choose k; it assumes round clusters of similar size; it is sensitive to scale and outliers; and it puts every point in some cluster, even if the point is an outlier.

2.10 DBSCAN

2.10.1 What it does

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) defines a cluster as a **dense region**: a place where points are packed closely together. It finds clusters of any shape, decides how many clusters there are by itself, and labels isolated points as **noise**.

2.10.2 How it does it

Two parameters: radius ε and `minPts`.

- A **core point** has at least `minPts` points within distance ε.
- A **border point** is within ε of a core point but is not core itself.
- A **noise point** is neither.

Clusters grow by chaining core points that are within ε of each other, then attaching their border points.

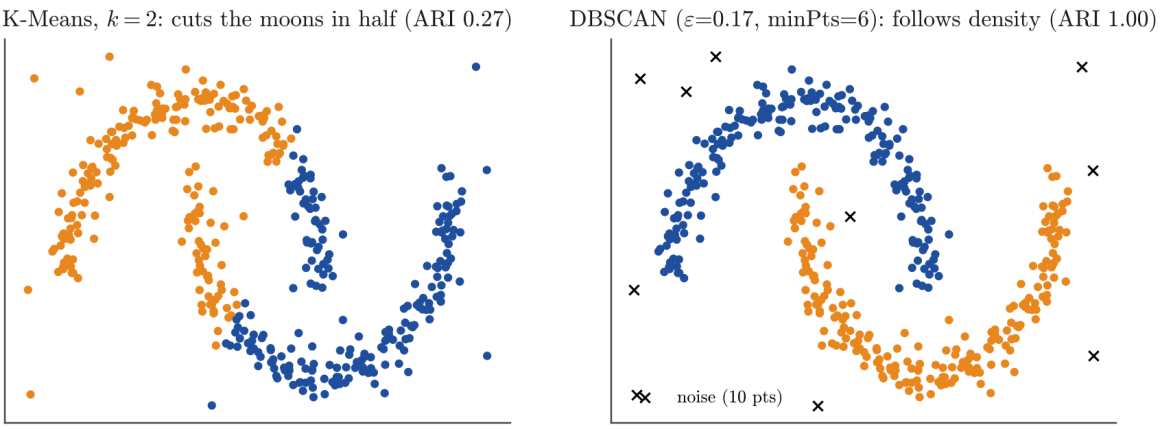


Figure 16: Two moons plus 25 random points. K-Means ($k = 2$) slices straight across both moons (ARI 0.27) because it can only make round clusters. DBSCAN recovers both moons exactly (ARI 1.00) and marks 10 isolated points as noise.

Choosing ε: plot every point's distance to its `minPts`-th nearest neighbour, sorted; pick ε at the knee. **Weakness:** one ε for the whole dataset, so clusters of very different densities are hard (HDBSCAN fixes this). **Uses:** geographic hotspots, anomaly detection, clusters with irregular shapes.

2.11 Hierarchical (Agglomerative) Clustering

2.11.1 What it does

Instead of one partition, hierarchical clustering builds the whole **family tree** of the data: which points merge first, which groups merge next, all the way to a single cluster. You choose the number of clusters afterwards by cutting the tree.

2.11.2 How it does it

Start with every point as its own cluster. Repeatedly merge the two closest clusters until one remains. "Closest" depends on the **linkage**:

Linkage	Distance between clusters A and B
Single	closest pair of points (finds chains; sensitive to noise)
Complete	farthest pair of points (compact clusters)
Average	mean distance over all pairs
Ward	merge that increases total within-cluster variance least (the usual default)

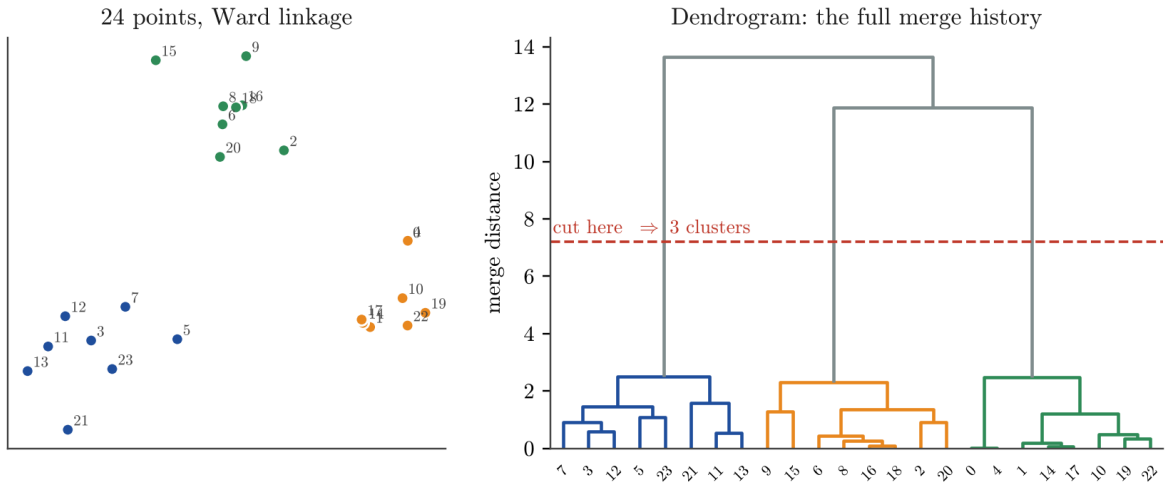


Figure 17: Ward clustering of 24 points. The dendrogram's height shows the distance at which groups merged. Long vertical gaps mean well-separated groups; cutting at the red line gives three clusters.

Strengths: no need to fix k in advance; the dendrogram itself is informative (taxonomies, gene expression, document hierarchies). **Weakness:** at least $O(n^2)$ memory and time, so it is impractical beyond tens of thousands of points.

2.12 Head-to-head: performance analysis

All eight models were run with near-default settings on three datasets, using 5-fold stratified cross-validation. Scaling was added where the algorithm needs it.

Table 1: Cross-validated accuracy (%, mean ± standard deviation over 5 folds). Bold = best on that dataset. Time is mean fit + predict per fold on digits, on one machine, so only the relative sizes mean anything.

Model	Breast cancer 569 rows, 30 feat. 2 classes	Wine 178 rows, 13 feat. 3 classes	Digits 1,797 rows, 64 feat. 10 classes	Time
Logistic regression	**97.9** ± 1.4	**98.3** ± 1.4	96.9 ± 1.0	53 ms
k-NN ($k = 5$)	96.5 ± 2.1	96.1 ± 1.3	97.7 ± 0.6	10 ms
Gaussian Naive Bayes	93.9 ± 2.0	97.2 ± 0.0	85.1 ± 2.5	7 ms
SVM (RBF kernel)	97.7 ± 0.7	**98.3** ± 1.4	**98.1** ± 0.7	98 ms
Decision tree	92.6 ± 2.2	92.7 ± 5.1	85.9 ± 2.0	21 ms
Random forest (300)	95.8 ± 1.7	97.8 ± 2.1	97.7 ± 0.5	1.1 s
XGBoost (300)	**97.9** ± 0.9	94.9 ± 2.8	96.5 ± 0.6	1.0 s
MLP (128–64)	97.5 ± 1.5	**98.3** ± 1.4	97.7 ± 0.8	1.3 s

2.12.1 What the table teaches

1. **On small, clean tables the strong models tie.** Logistic regression, SVM, random forest, XGBoost and the MLP are all within 1–2 points, about one standard deviation. A 0.4-point gap over 5 folds is not evidence that one model is better. Report the spread, not just the mean.

2. **Simple models are competitive.** Logistic regression ties for best on two of three datasets and trains in milliseconds. Always run a simple baseline first; it tells you whether the complex model is earning its cost.
3. **Broken assumptions show up.** Naive Bayes (independence) and a single tree (variance) both collapse to about 85% on digits, where neighbouring pixels are highly correlated.
4. **Ensembles fix variance.** Single tree → random forest: 85.9% → 97.7% on digits.
5. **Deep learning is not automatically better.** The MLP takes about 25 times longer than logistic regression on digits for a 0.8-point gain. Neural networks win on large amounts of raw, high-dimensional data (images, audio, text), which is where the course goes next.
6. **No model wins everywhere** (the "no free lunch" theorem). XGBoost is best on breast cancer but weakest of the strong models on wine, which has only 178 rows.

2.13 Choosing an algorithm

Situation	Start with
Need to explain every prediction (credit, medicine)	logistic / linear regression, a shallow tree
Tabular data, want the best accuracy	XGBoost / LightGBM, then a random forest
Very little data, many text features	Naive Bayes, linear SVM, logistic regression
Small or medium data with complex boundaries	SVM with RBF kernel
"Find similar items", quick baseline	k-NN
Group customers, known k, round groups	K-Means
Odd-shaped groups, outliers expected	DBSCAN
Want the full grouping structure, small n	hierarchical clustering
Images, audio, raw text, very large data	neural networks (Section 7)

3 Data Augmentation

3.1 Why we need it: labelled data is the bottleneck

Models need many labelled examples, and labels are expensive. Every label needs a person, medical labels need specialists, and rare classes (fraud, rare diseases) have few examples no matter how much you spend. Our MNIST MLP has 109,386 parameters but the baseline notebook trains it on 5,000 images, about 22 parameters per example. That is plenty of room to memorise.

Definition: Data augmentation

Create new training examples by applying transformations T that **change the input but not the label**:

$$(\mathbf{x}, y) \longrightarrow (T(\mathbf{x}), y).$$

Each transformation is a claim: *"this change does not affect the answer."* Applied randomly every epoch, the model sees a slightly different version of every image each time.

3.2 Image augmentation

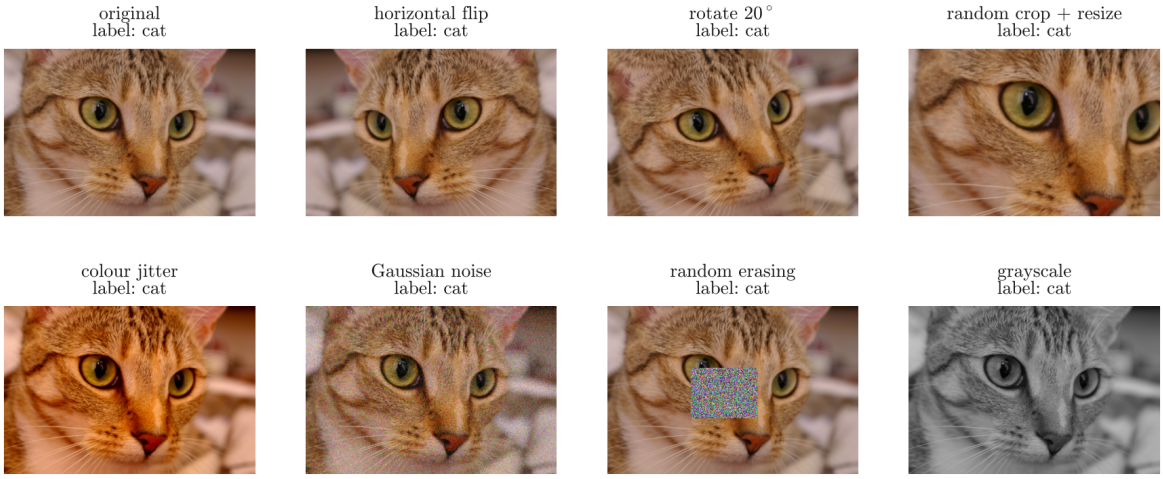


Figure 18: One photo, eight training examples. Every version is still labelled "cat".

Transform	What it teaches the model
Horizontal flip	left vs. right does not matter (but not for text or road signs)
Rotation	tilted camera or scanned page
Random crop + resize	object position and scale vary
Colour jitter	lighting and camera differences
Gaussian noise	sensor noise, low light
Random erasing / cutout	occlusion: do not rely on a single region
Grayscale	shape matters more than colour

3.3 Text augmentation

Text is harder: change one word and the meaning can flip. The four cheap "EDA" operations (Wei & Zou, 2019) and one expensive one, applied to a negative movie review:

Operation	Result
Original	*The plot was boring and the acting was terrible.* (negative)
Synonym replacement	*The plot was **dull** and the acting was **awful**.*
Random insertion	*The plot was boring and the **truly** acting was terrible.*
Random swap	*The **acting** was boring and the **plot** was terrible.*
Random deletion	*The plot boring and the acting was terrible.*
Back-translation (EN→FR→EN)	*The story was boring and the performances were terrible.*

Your Text Augmentation notebook implements random swap, synonym replacement, character-level typos, back-translation (mocked with a fixed translation) and entity replacement, and applies augmentation to a whole labelled dataset.

3.4 The golden rule: would a human give it the same label?

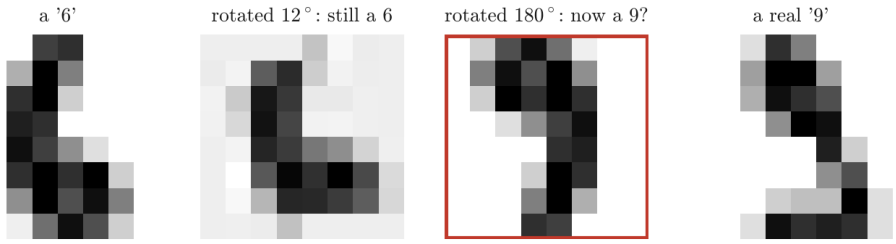


Figure 19: A 12° rotation keeps a "6" a "6". A 180° rotation turns it into something that looks like a "9", so the transform has silently changed the label.

Safe: label preserved	Unsafe: label changes
Flipping a photo of a cat or a car	Flipping text, or a "left turn" sign
Rotating a handwritten digit by 10°	Rotating a "6" by 180°
"boring" → "dull" in a review	"not good" → "good" by deleting "not"
Brightness changes on street scenes	Colour shifts when colour is the label (ripe vs. unripe fruit)

3.5 Where augmentation lives in the pipeline

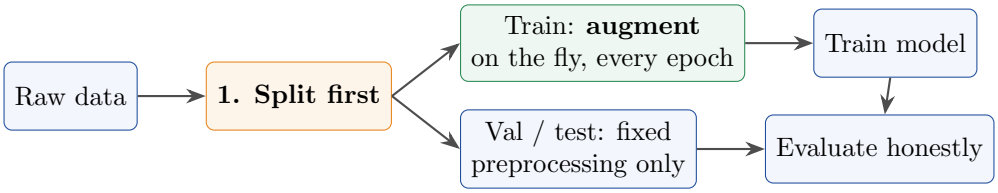


Figure 20: Split before augmenting. If augmented copies of a test image end up in training, the test score measures memory, not generalisation.

3.6 Does it work? A measured experiment

We trained the same MLP (128–64 hidden units) on 8×8 handwritten digits with and without augmentation (random 1-pixel shifts and ±12° rotations, 4 augmented copies per image), averaging 5 random seeds. We tested on the normal centred test set and on a *shifted* copy of the same test set in which each digit is moved up to one pixel, as a scanner or camera might do in deployment.

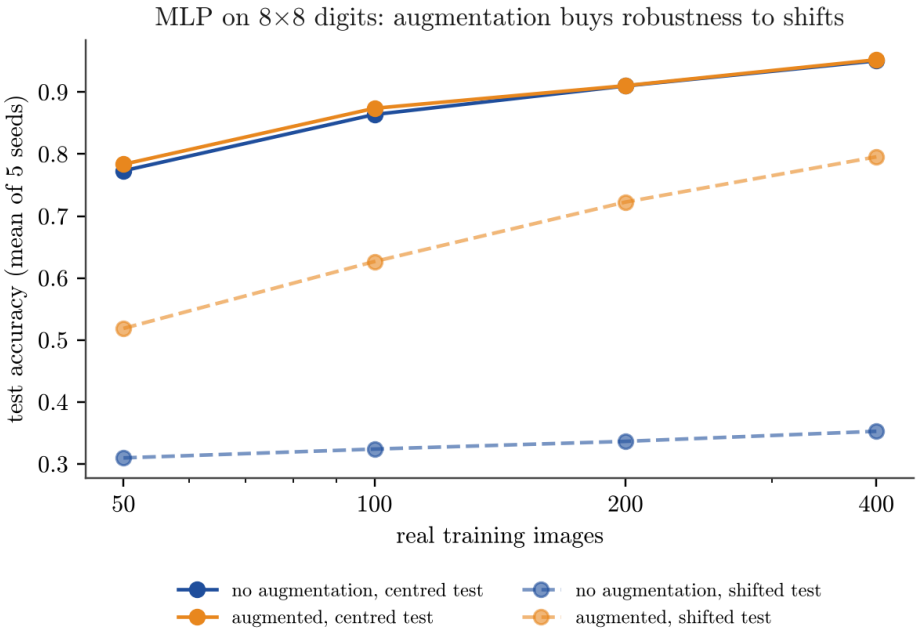


Figure 21: Test accuracy vs. number of real training images.

Real training images	Centred test set		Shifted test set	
	no aug.	augmented	no aug.	augmented
50	77.3%	78.4%	31.0%	51.9%
100	86.4%	87.4%	32.4%	62.7%
200	91.0%	91.0%	33.7%	72.3%
400	95.0%	95.2%	35.3%	79.6%

3.6.1 Analysis

- **On the centred test set augmentation barely matters** (+0 to +1 point). The test digits look like the training digits, so there is little new to learn.
- **On shifted digits the un-augmented MLP collapses** to 31–35%, even though the shift is only one pixel. With 400 images, augmentation lifts it from 35% to 80%.
- **Lesson:** augmentation does not just add data. It teaches **invariance**, telling the model what should not matter. Its value depends on whether the variation you add actually shows up in deployment.
- **Why the MLP is so fragile:** it flattens the image into 64 independent numbers and has no idea that pixel 10 and pixel 11 are neighbours. A one-pixel shift moves every value to a different input. Convolutional networks (next chapter) build shift tolerance into the architecture.

Augmented copies are not free data

Five augmented copies of one image are highly correlated: they are worth more than one image but much less than five new real images. And augmenting a dataset that is mislabelled or biased copies the problem along with the data.

Tabular data: augmentation is less common. The main tool is oversampling the rare class, either by duplicating rows or with SMOTE, which creates synthetic minority points by interpolating between neighbours. Use it on the training fold only.

4 Train, Validation and Test Splits

4.1 Why three sets?

The exam analogy

Training set = homework problems with solutions; you learn from them. **Validation set** = mock exams; you use them to decide how to study (which model, how much regularisation, when to stop). **Test set** = the final exam, taken *once*. If you peek at the final and adjust your studying, your grade no longer measures what you know.

Set	Used for	Typical share
Train	fitting the parameters (weights, splits, centroids)	60–80%
Validation	choosing hyper-parameters, models, features; early stopping	10–20%
Test	one final, unbiased estimate of real-world performance	10–20%

If you choose between 50 models by their test score, the winner's test score is optimistic: you picked it partly because it got lucky on those particular test examples. That is why hyper-parameter search uses the validation set, and the test set is touched once at the end.

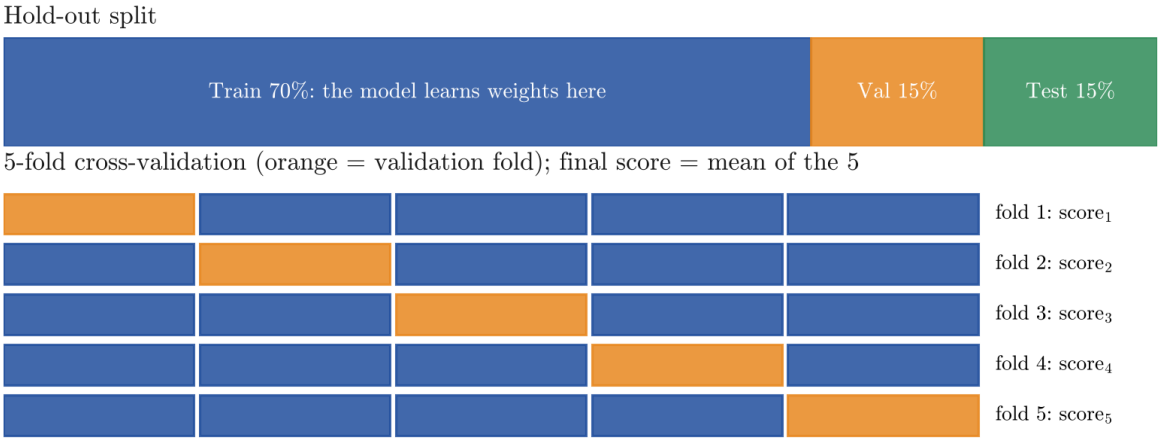


Figure 22: Top: a single hold-out split. Bottom: 5-fold cross-validation. Every row is used for validation exactly once, and the five scores are averaged. Table 1 was computed this way.

4.2 Cross-validation

With small datasets a single validation split is noisy: 15% of 178 wine samples is only 27 rows. **k-fold cross-validation** splits the training data into k folds, trains k times (each time holding out a different fold) and averages. The standard deviation across folds tells you how much to trust a difference between models.

4.3 Doing it right

- **Stratify** classification splits so every set has the same class proportions. With 1% fraud, a random split can leave the test set with almost no fraud cases.
- **Split by time** for anything that is forecast: train on January–June, validate on July, test on August. A random split lets the model "see the future".
- **Split by group** when rows are related: all images from the same patient go in the same set; otherwise the model recognises the patient, not the disease.
- **Fit preprocessing on training data only.** Compute the mean and standard deviation for scaling from the training set, then apply those numbers to validation and test. A scikit-learn `Pipeline` does this automatically inside cross-validation.

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 1. Hold out a test set once, stratified by class
X_trval, X_test, y_trval, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 2. Model selection with 5-fold CV on the remaining 80%.
# The scaler is refit inside every fold, so nothing leaks.
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
scores = cross_val_score(model, X_trval, y_trval, cv=5)
print(f"CV accuracy: {scores.mean():.3f} +/- {scores.std():.3f}")

# 3. Final check, once
model.fit(X_trval, y_trval)
print("Test accuracy:", model.score(X_test, y_test))
```

Data leakage: the most common way to fool yourself

Leakage is any route by which information from the validation or test data reaches training. Classic cases: scaling the whole dataset before splitting; augmenting before splitting; duplicate rows across sets; and features that are only known after the outcome ("number of follow-up visits" used to predict readmission). Leakage gives great validation scores and a model that fails in production.

5 Loss Functions, Learning and Optimisation

5.1 Loss: a single number for "how wrong"

Training needs one number to minimise. The loss function is how you tell the algorithm what kind of mistake matters.

Loss	Formula (one example)	Task	Behaviour, with a worked number
MSE	$(y-\hat{y})^2$	regression	big errors hurt quadratically: $y=3, \hat{y}=5 \Rightarrow 4$
MAE	$\lvert y-\hat{y}\rvert$	regression	robust to outliers: same case $\Rightarrow 2$
Binary CE	$-[y\log\hat{p}+(1-y)\log(1-\hat{p})]$	binary class.	$y=1$: $\hat{p}=0.9 \Rightarrow 0.105$; $\hat{p}=0.1 \Rightarrow 2.303$
Categorical CE	$-\log \hat{p}_{\text{true class}}$	multi-class	true class gets $0.7 \Rightarrow 0.357$
Hinge	$\max(0,\, 1-y\,z)$, $y\in\{\pm 1\}$	SVM	zero once a point is beyond the margin

Notice the cross-entropy numbers: being 90% sure and right costs 0.105; being 90% sure and wrong costs 2.303, about 22 times more. Cross-entropy punishes confident mistakes hard.

5.2 Learning: gradient descent

The gradient ∇L points uphill. Gradient descent takes a step the other way:

$$\boldsymbol{\theta} \;\leftarrow\; \boldsymbol{\theta} - \eta\,\nabla_{\boldsymbol{\theta}} L.$$

The **learning rate** η is the step size, and it is the most important hyper-parameter in deep learning.

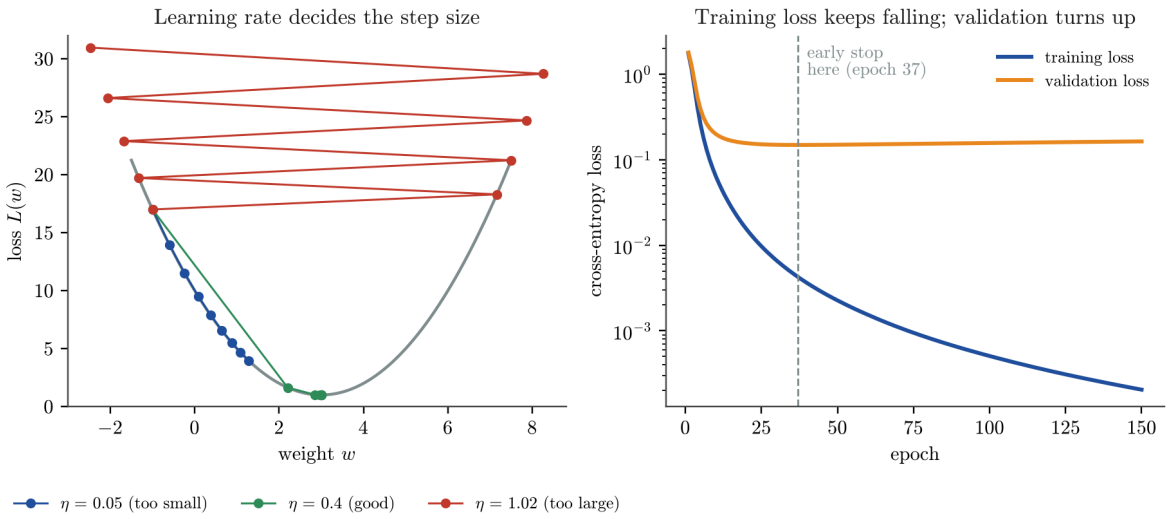


Figure 23: Left: gradient descent on $L(w)=(w-3)^2+1$ from $w=-1$. Too small a step crawls, a good step converges in a few moves, and too large a step overshoots and diverges. Right: an over-sized MLP (256–256) trained on only 300 digit images. Training loss keeps falling; validation loss bottoms out at epoch 37 and then rises: overfitting.

5.2.1 Batch, mini-batch and stochastic

- **Batch gradient descent:** the gradient uses all n examples. Accurate but slow per step.
- **Stochastic gradient descent (SGD):** one example per step. Fast and noisy; the noise can help it escape poor regions.
- **Mini-batch:** 32–512 examples per step. The standard compromise, and it suits GPUs.

An **epoch** is one full pass over the training data. With 5,000 images and batch size 64, one epoch is $\lceil 5000/64 \rceil = 79$ updates (**iterations**). Optimisers such as **Adam** adapt the step size for each parameter and are the usual default for neural networks (your MNIST notebook uses Adam with $\eta = 0.001$).

5.3 Reading learning curves

What you see	Diagnosis	What to try
Both losses high and flat	underfitting (high bias)	bigger model, better features, train longer, higher η
Training low, validation high and rising	overfitting (high variance)	more data, augmentation, regularisation, early stopping
Loss jumps around or explodes	learning rate too high	lower η, gradient clipping
Loss falls painfully slowly	learning rate too low	raise η, use Adam

In the right panel of Figure 23, by epoch 150 the training loss is 0.0002 (the model has memorised the 300 images) while validation loss has risen from its best of 0.149 to 0.164. **Early stopping** keeps the weights from the best validation epoch.

5.4 Regularisation

Regularisation adds a penalty for complexity to the loss:

$$L_{\text{total}} = L_{\text{data}} + \lambda\,\Omega(\boldsymbol{\theta}), \qquad \Omega = \textstyle\sum_j w_j^2 \text{ (L2 / ridge)} \quad \text{or} \quad \sum_j |w_j| \text{ (L1 / lasso)}.$$

L2 shrinks all weights a little; L1 pushes some exactly to zero, selecting features. Other regularisers you have now seen: limiting tree depth, k in k-NN, C in SVMs, augmentation and early stopping. Dropout, for neural networks, comes in Chapter 1.

6 Evaluating Classifiers: Confusion Matrices and Metrics

6.1 The confusion matrix

Accuracy compresses everything into one number. The **confusion matrix** shows *which* mistakes the model makes. Fix one class as "positive" (the thing you are trying to catch, e.g. malignant):

		Predicted	
		Positive	Negative
Actual	Positive	**TP** true positive	**FN** false negative (miss)
	Negative	**FP** false positive (false alarm)	**TN** true negative

6.2 The metrics, from the matrix

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad \text{overall fraction correct}$$

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{of everything flagged, how much was real?}$$

$$\text{Recall (sensitivity)} = \frac{TP}{TP + FN} \qquad \text{of everything real, how much did we catch?}$$

$$\text{Specificity} = \frac{TN}{TN + FP} \qquad \text{of the negatives, how many did we leave alone?}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \qquad \text{harmonic mean; low if either is low}$$

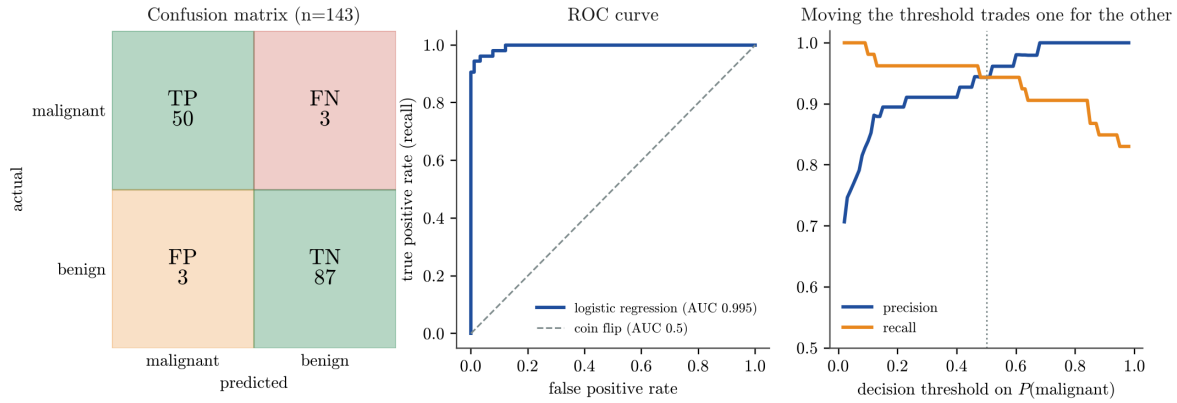


Figure 24: Logistic regression on the breast-cancer test set (143 tumours, 53 malignant). Left: the confusion matrix with malignant as the positive class. Middle: the ROC curve. Right: precision and recall as the decision threshold moves.

Worked example: reading the breast-cancer matrix

$TP = 50$, $FN = 3$, $FP = 3$, $TN = 87$, $n = 143$.
Accuracy $= (50 + 87)/143 = 95.8\%$. Precision $= 50/53 = 94.3\%$. Recall $= 50/53 = 94.3\%$. $F_1 = 94.3\%$.
In words: of 53 malignant tumours the model caught 50 and **missed 3**; it also raised 3 false alarms on benign tumours. In screening, the 3 misses are what matters, which leads to thresholds.

6.3 Thresholds: precision vs. recall

A classifier outputs a probability; we choose where to cut. The default 0.5 is arbitrary. Lowering the threshold to 0.2 on the breast-cancer model:

Threshold	Recall	Precision	Missed cancers (FN)	False alarms (FP)
0.5	94.3%	94.3%	3	3
0.2	96.2%	89.5%	2	6

One fewer missed cancer, at the cost of three more benign patients sent for a biopsy. Whether that is worth it is a medical and business decision, not a statistical one. The model only shows you the trade-off.

ROC and AUC. The ROC curve plots recall (true positive rate) against false positive rate for every threshold. The area under it (AUC) is the probability that a random positive gets a higher score than a random negative: 0.5 is a coin flip, 1.0 is perfect. Our model scores 0.995, so its *ranking* is nearly perfect even though a few tumours fall on the wrong side of 0.5.

6.4 The accuracy paradox

Out of 10,000 card transactions, 100 are fraud. A "model" that answers *not fraud* every time scores **99% accuracy** and has **0% recall**: it catches nothing. On imbalanced data, report precision, recall, F_1 or PR-AUC, never accuracy alone.

6.5 Which metric when?

Situation	Optimise	Why
Missing a positive is costly (cancer, fraud, safety)	recall	a miss is worse than a false alarm
False alarms are costly (spam folder, legal flags)	precision	a wrong flag harms a real user
Both matter, classes imbalanced	F_1, PR-AUC	a single number sensitive to both
Ranking quality, threshold chosen later	ROC-AUC	threshold-independent
Balanced classes, equal costs	accuracy	easy to read
Multi-class	macro-F_1 (classes equal) or micro / accuracy (rows equal)	macro exposes weak rare classes

Regression metrics: MAE (average miss, in the target's units), RMSE (punishes large misses more) and R^2 (share of variance explained). On the diabetes data, RMSE dropped from 70.5 (predict the mean) to 56.4 with ten features.

7 Pivoting to Deep Learning: the Perceptron and the MLP

7.1 Why classical ML is not enough

Every model so far depends on **features a human designed**. For a spreadsheet that is fine: the columns already mean something. For a 224×224 colour photo there are $224 \times 224 \times 3 = 150{,}528$ raw numbers, and no single pixel means anything. Before deep learning, vision experts spent years hand-crafting features (edges, corners, textures). **Deep learning learns the features themselves**, layer by layer, from raw data. It starts with one artificial neuron.

7.2 The perceptron (Rosenblatt, 1958)

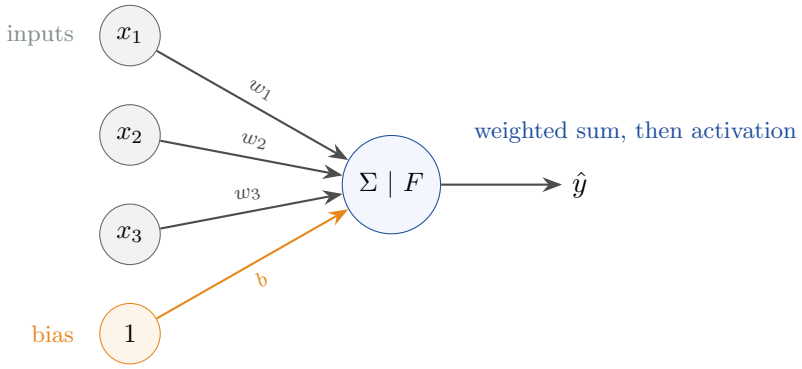


Figure 25: The perceptron: $\hat{y} = F(\mathbf{w}^\top \mathbf{x} + b)$, where F is a step function.

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \text{step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Learning rule: for each misclassified example, $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}$ and $b \leftarrow b + \eta\,(y - \hat{y})$. If the classes can be separated by a line, this rule is guaranteed to find one.

You have already met this neuron

Swap the step for a sigmoid and the perceptron *is* logistic regression (Section 2.2). A neuron is a linear model with a squashing function. Everything new in deep learning comes from **stacking** them.

7.3 The XOR problem

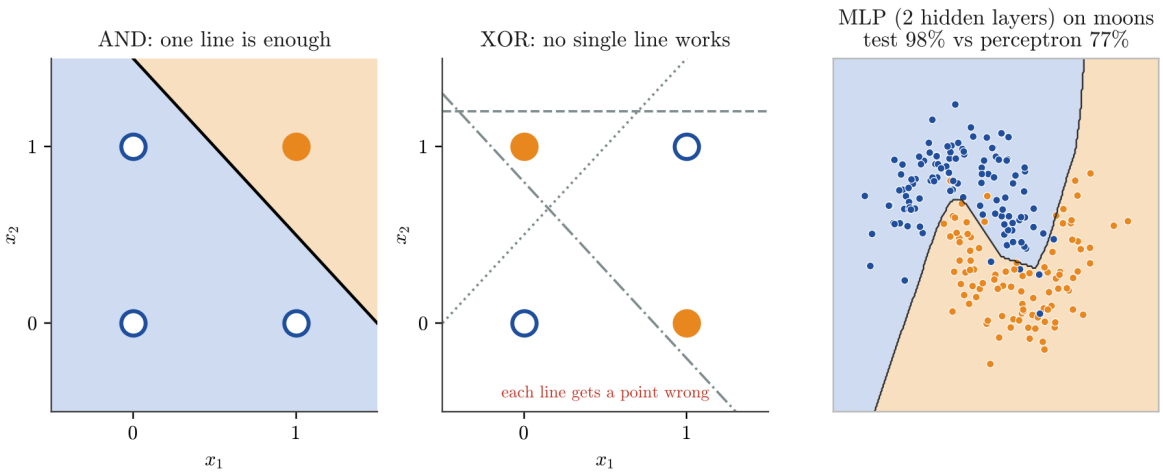


Figure 26: Left: AND is linearly separable, and $w_1 = w_2 = 1$, $b = -1.5$ solves it. Middle: XOR is not; every straight line misclassifies at least one point. Right: a two-hidden-layer MLP on noisy moons reaches 98% test accuracy, vs. 77% for a perceptron.

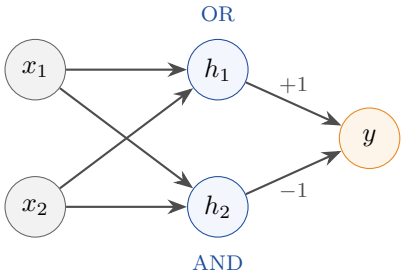
Minsky and Papert's 1969 book *Perceptrons* proved that a single-layer perceptron cannot learn XOR. It was widely read as a verdict on neural networks in general, and funding dried up for

over a decade. The fix was already known in principle: add a hidden layer. What was missing was a way to train one.

7.4 From perceptron to multi-layer perceptron

Two hidden units solve XOR. One computes OR, the other AND, and the output fires for "OR but not AND":

$$h_1 = \text{step}(x_1 + x_2 - 0.5), \quad h_2 = \text{step}(x_1 + x_2 - 1.5), \quad y = \text{step}(h_1 - h_2 - 0.5).$$



x_1	x_2	h_1 (OR)	h_2 (AND)	y
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Each hidden unit draws one line. The output unit combines the two lines into a region that no single perceptron could draw. That is the whole idea of depth: **each layer builds new features out of the previous layer's features.**

7.5 Two ingredients that made it trainable

1. **A differentiable activation.** The step function has zero gradient almost everywhere, so gradient descent gets no signal. Sigmoid (1986), and later ReLU (about 2010), fixed that.
2. **Backpropagation** (Rumelhart, Hinton & Williams, 1986): the chain rule, applied layer by layer from the output back to the input, gives every weight its share of the blame for the error.

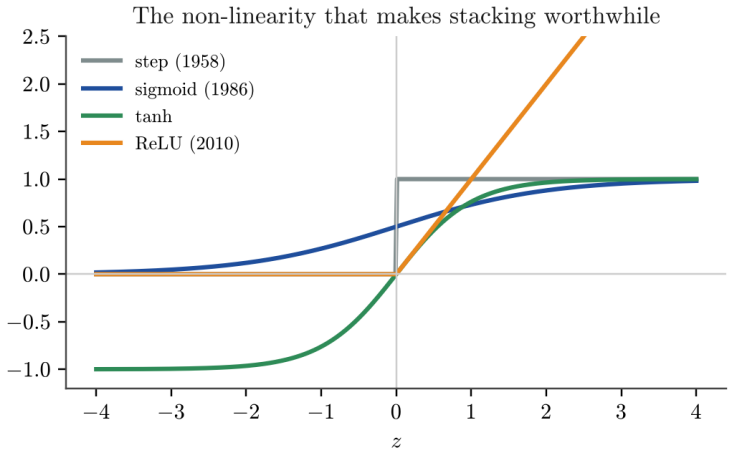


Figure 27: Activation functions. Without a non-linearity, stacking layers is pointless: two linear layers collapse into one, $W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$.

7.6 The MLP you will train: 784 → 128 → 64 → 10

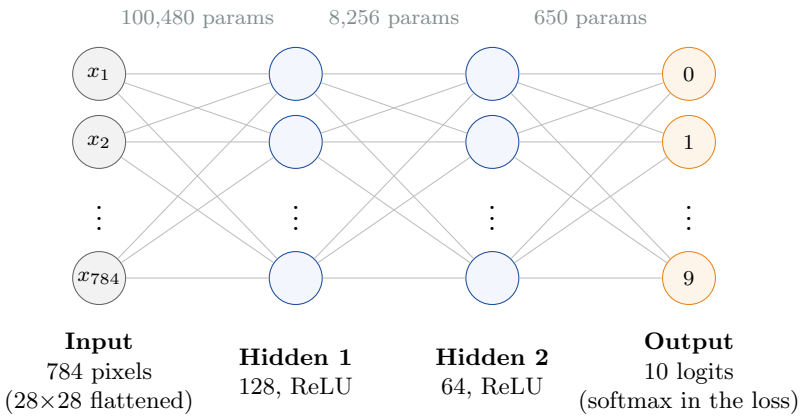


Figure 28: The MNIST MLP from the course notebook.

Parameters per layer are inputs × outputs + biases: $784 \times 128 + 128 = 100{,}480$; $128 \times 64 + 64 = 8{,}256$; $64 \times 10 + 10 = 650$; total **109,386**. In PyTorch:

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(), # (B,1,28,28) -> (B,784)
            nn.Linear(784, 128), nn.ReLU(), # hidden layer 1
            nn.Linear(128, 64), nn.ReLU(), # hidden layer 2
            nn.Linear(64, 10)) # logits; CrossEntropyLoss applies softmax
    def forward(self, x):
        return self.net(x)

model, loss_fn = MLP(), nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(5):
    for images, labels in train_loader: # mini-batches of 64
        loss = loss_fn(model(images), labels) # 1. predict 2. measure
        opt.zero_grad(); loss.backward() # 3. backprop gradients
        opt.step() # 4. update
```

Every idea from these notes is in that short block: a model family (stacked linear layers with ReLU), a loss (cross-entropy), an optimiser (Adam, mini-batches, epochs) and the four-step training loop. What is new is the depth, which lets the network learn its own features.

What an MLP still cannot see

Flattening throws away the 2D layout: to the MLP, two neighbouring pixels are no closer than two pixels at opposite corners. The augmentation experiment showed the cost, with accuracy collapsing to about 33% under one-pixel shifts. Convolutional neural networks fix this by building locality and shift tolerance into the architecture.

Chapter 1 covers the MLP in full: forward propagation with numbers, backpropagation step by step, every activation function, and softmax with cross-entropy.

8 Summary

Key takeaways

1. **ML learns $f(\mathbf{x}) \approx y$ from examples.** Every method is a model family plus a loss plus an optimiser.
2. **The goal is generalisation.** Training accuracy is not the score; held-out performance is.
3. **Classical models still rule tabular data.** Start with a simple baseline (logistic or linear regression), then try boosted trees. On our three datasets the strong models were within about one standard deviation of each other.
4. **Each algorithm encodes an assumption**: linearity (regression), similarity means same label (k-NN), independence (Naive Bayes), wide margins (SVM), axis-aligned rules (trees), round clusters (K-Means), density (DBSCAN). It works when its assumption fits the data.
5. **Augmentation teaches invariance** and only helps if the variation shows up in deployment.
6. **Split first, test once.** Leakage is the most common way to fool yourself.
7. **Pick the metric from the cost of each mistake.** Accuracy misleads on imbalanced data.
8. **An MLP is perceptrons + non-linearity + backpropagation**, and it learns its own features.

8.1 Practice questions

1. A model scores 99% on training data and 71% on validation. Name the problem and three fixes.
2. Fit $\hat{y} = wx + b$ by hand to $(1, 3), (2, 5), (3, 7)$. What are w, b and the MSE?
3. For logistic regression with $w = 0.8$ and $b = -2$, what is $P(y = 1)$ at $x = 4$? By what factor do the odds change when x increases by 1?
4. Why must features be standardised for k-NN and SVMs but not for random forests?
5. In Table 1, Naive Bayes is competitive on wine but not on digits. Explain using its independence assumption.
6. Compute the Gini impurity of a node with 30 positives and 10 negatives.
7. Give one dataset where K-Means fails and DBSCAN succeeds, and one where DBSCAN struggles.
8. You augment digit images with vertical flips. What could go wrong?
9. A colleague standardises the full dataset and then splits it into train and test. What is wrong, and why does it inflate the test score?
10. A fraud model at threshold 0.5 has $TP = 40$, $FN = 60$, $FP = 10$, $TN = 9{,}890$. Compute accuracy, precision and recall. Is this a good model? Which way should the threshold move?

11. Show that two stacked linear layers without an activation are equivalent to one linear layer.
12. How many parameters does a $784 \to 256 \to 10$ MLP have?

Reproducibility. All figures and numbers: `python make_figures.py` (scikit-learn 1.9, XGBoost 3.4, NumPy 2.5). Datasets: scikit-learn's bundled breast cancer, wine, digits, iris and diabetes datasets; the cat photo is scikit-image's `chelsea` sample.